

Общие сведения:

Проект представляет собой roguelike игру с ASCII графикой на c++. Базой для проекта является BearLibTerminal, который предоставляет удобное считывание данных с клавиатуры и поддерживает работу со своим искусственным терминалом.

Задачи проекта сделать игру достаточно расширяемой, учитывая тот факт, что излишняя модульность может помешать скорости создания проекта и нужно искать компромисс. Так же важна простота использования кода при добавлении новых фичей. Весь проект компилируется на c++ и всё необходимое для запуска уже находится в репозитории.

Для компиляции используется команда

```
g++ main.cpp -o my_game -ldl -L. -lBearLibTerminal -Wl,-rpath=.
```

Для тестирования....

Architectural drivers:

Основной архитектурной идеей проекта является факт того, что современные игры имеют всё больше легаси кода и используют слишком много абстракций прежде чем появятся у конкретного пользователя.

Использование C++ помогает игре использовать то, что использовали разработчики у истоков создания игр и игровых движков. Увеличивая портируемость за счёт использования ASCII графики.

Изучение паттернов и методов их применения также является одной из главных задач данного проекта. Это позволяет как и ускорить разработку так и упростить придумывание архитектуры данного проекта.

Роли и случаи использования:

Главной ролью проекта является не только развлечение пользователя, но и предоставление платформы для удобной доработки игры под свои хотелки и желания. С чем и будет помогать структура проекта и конечный пользователь сможет быстро разобраться в том как работает игра и где можно удобно устроиться(использование паттернов).

Описание типичного пользователя:

Типичный пользователь геймер либо программист.

Первый хочет играть в игру, в которой имеется цельная система, разработчики которой способны быстро фиксировать баги.

Второй хочет иметь возможность для быстрой доработки игры с целью реализации своих идей для роулайк систем.

Диаграмма классов:

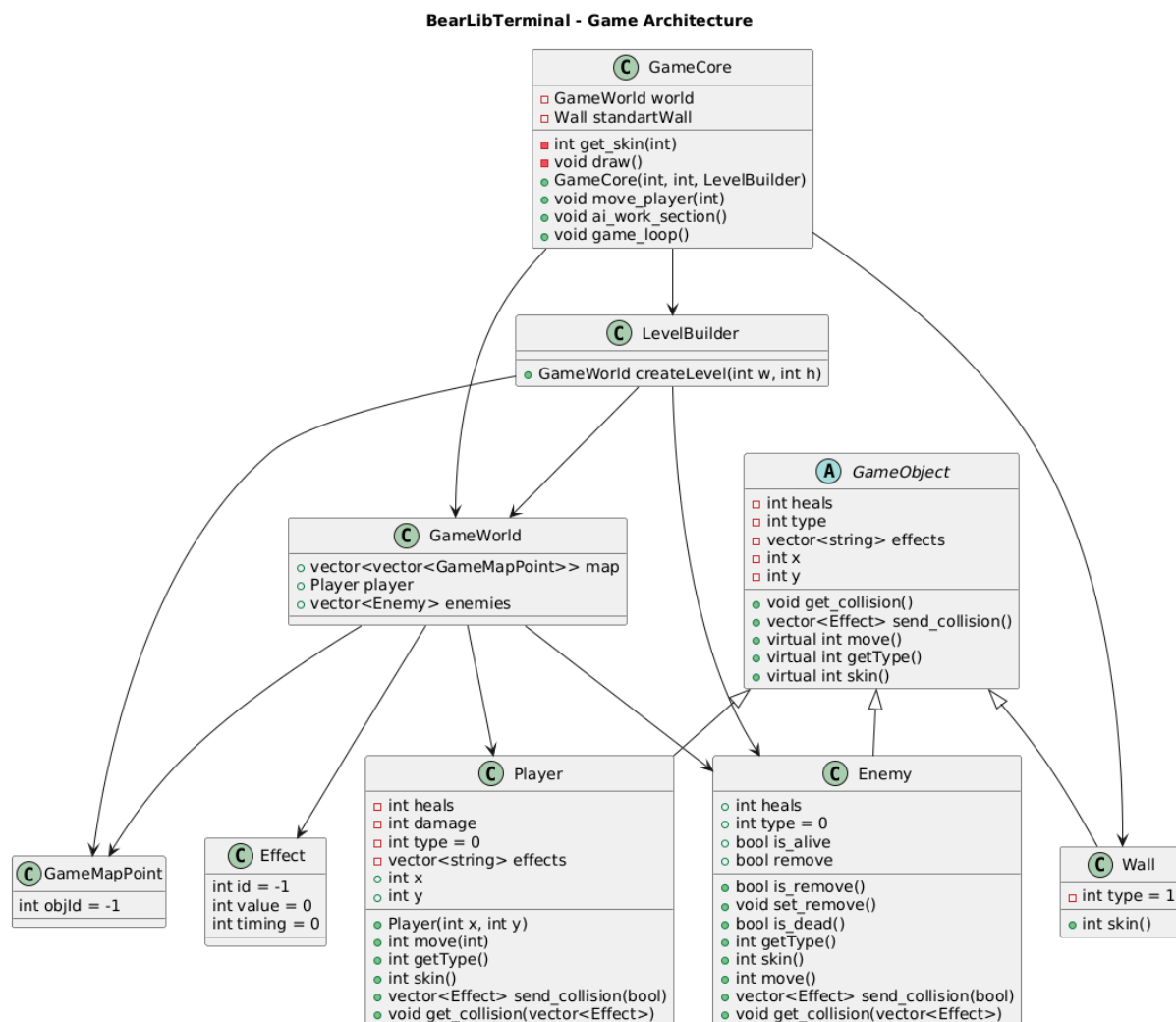


Диаграмма компонентов:

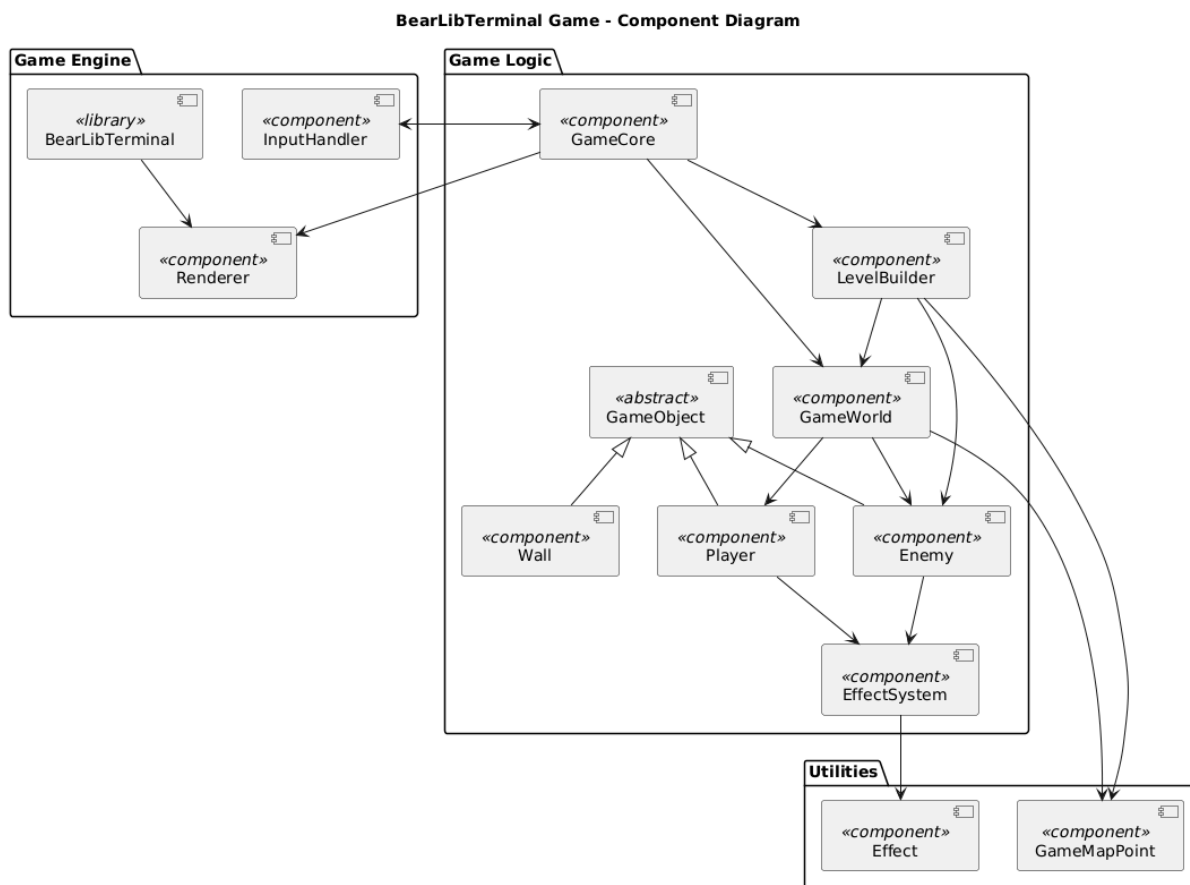


Диаграмма последовательностей:

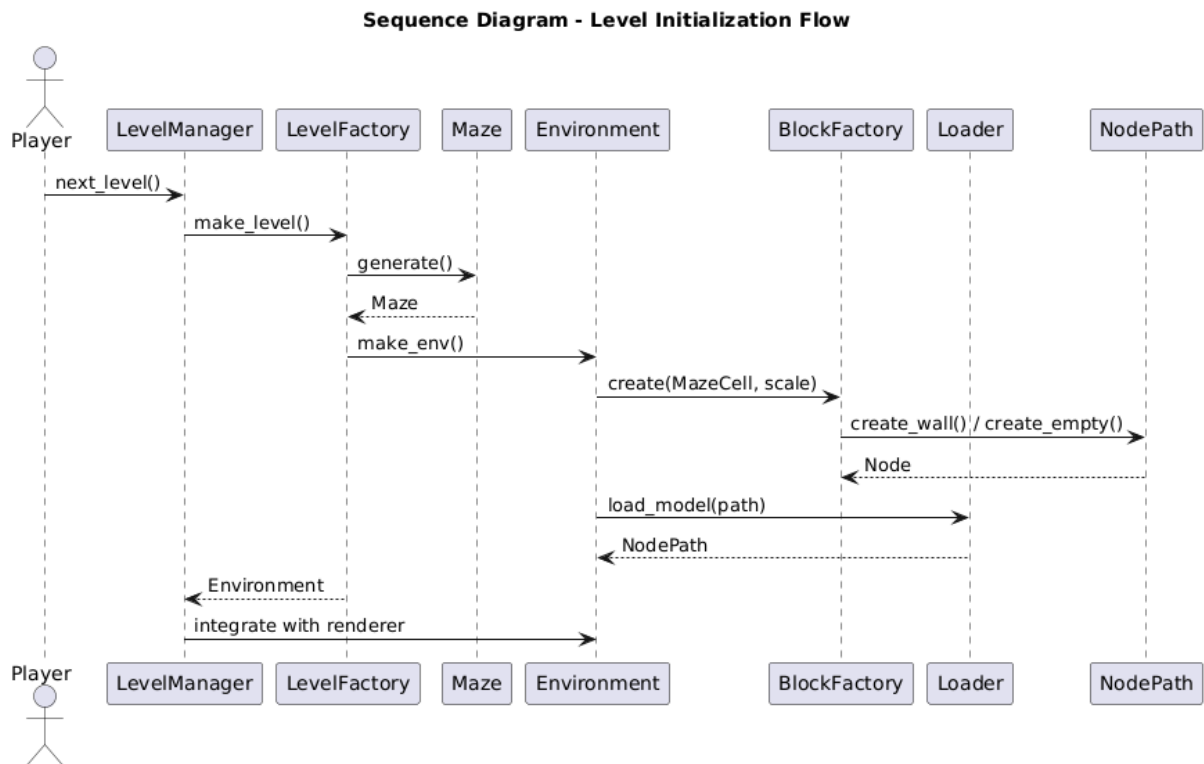


Диаграмма состояний движка уровня:

Level Lifecycle State Machine

